

Semana 11: Archivos y Pandas — Del CSV al análisis de datos

Guía completa de la sesión

Visión general

Duración total	3 horas (1.5 h cátedra con live coding + 1.5 h laboratorio en Colab)
Objetivo central	Que los estudiantes carguen archivos CSV con Pandas, exploren DataFrames, y apliquen las funciones de la Semana 10 sobre datos ecológicos reales
Idea ancla	En la Sem. 9 vieron que una lista de diccionarios es una tabla: cada dict = fila, cada clave = columna. Un DataFrame de Pandas es exactamente eso — pero con superpoderes: filtrado, agrupación, estadísticas, todo en una línea
Prerrequisito	Sem. 9 (listas, diccionarios, lista de dicts), Sem. 10 (funciones)
Materiales	Proyector, Colab, WiFi, CSVs (bosque_valdiviano_simple.csv, camaras_trampa_simulado.csv)
Datasets	Ambos CSVs compartidos en Drive o URL pública

PARTE 1: CÁTEDRA CON LIVE CODING (90 minutos)

Bloque A — El puente: de la lista de dicts al DataFrame (10 min)

Proyectar el código de la Sem. 9:

```
# Semana 9: lista de diccionarios
arboles = [
    {"especie": "N. dombeyi", "dap": 45.3, "altura": 22.1, "parcela": 1},
    {"especie": "D. winteri", "dap": 25.1, "altura": 15.8, "parcela": 1},
    {"especie": "A. luma", "dap": 18.7, "altura": 12.3, "parcela": 2},
]
```

```
# Para calcular el DAP promedio:
daps = [a["dap"] for a in arboles]
promedio = sum(daps) / len(daps)
```

“Funciona. Pero para 100 árboles con 10 columnas, escribir list comprehensions para cada cálculo es tedioso. ¿Existe algo mejor?”

```
import pandas as pd
```

```
# MISMO dato, formato Pandas
df = pd.DataFrame(arboles)
print(df)
print(df["dap"].mean()) # una línea
```

“El DataFrame es la lista de diccionarios con superpoderes.”

Bloque B — Cargar datos con Pandas (10 min)

Desde un CSV

```
import pandas as pd

# Opción 1: desde Google Drive
from google.colab import drive
drive.mount('/content/drive')
df = pd.read_csv("/content/drive/MyDrive/CBIT010_Archivos_ayudantia/Semana11/bosque_valdiviano_simple.csv")

# Opción 2: desde URL
# df = pd.read_csv("https://URL_DIRECTA_AL_CSV")
```

Primera exploración

```
print(df.shape)          # (100, 4) → 100 filas, 4 columnas
print(df.columns)        # Index(['especie', 'dap_cm', 'altura_m', 'parcela'])
print(df.dtypes)         # tipos de cada columna

df.head()                # primeras 5 filas
df.tail(3)               # últimas 3 filas
df.info()                # resumen completo
```

“Fíjense: Pandas detectó automáticamente que **dap_cm** es float y **parcela** es int. No necesitaron convertir a mano como con `csv.DictReader`.”

Bloque C — Anatomía del DataFrame (15 min)

Columnas (Series)

```
# Seleccionar una columna → devuelve una Serie
daps = df["dap_cm"]
print(type(daps))        # pandas.core.series.Series
print(daps.head())

# Estadísticas de una columna
print(daps.mean())       # promedio
print(daps.median())     # mediana
print(daps.std())        # desviación estándar
print(daps.min())        # mínimo
print(daps.max())        # máximo
```

Resumen estadístico

```
df.describe()            # estadísticas de TODAS las columnas numéricas
```

“Una línea para obtener *n*, promedio, std, min, Q1, mediana, Q3, max de cada variable numérica. Eso antes les tomaba 30 líneas con bucles.”

Seleccionar múltiples columnas

```
# Subconjunto de columnas
df[["especie", "dap_cm"]]

# Seleccionar filas por índice
df.iloc[0]               # primera fila (por posición)
df.iloc[0:5]             # filas 0 a 4
```

Bloque D — Filtrar datos (15 min)

Callback Sem. 9: “En la Sem. 9, para filtrar árboles grandes escribían: `[a for a in arboles if a['dap'] > 30]`. En Pandas es una línea.”

Filtrado con condiciones booleanas

```
# ¿Cuáles árboles tienen DAP > 30?
mascara = df["dap_cm"] > 30
print(mascara.head())    # True/False para cada fila

# Aplicar la máscara
grandes = df[df["dap_cm"] > 30]
print(f"Árboles con DAP > 30: {len(grandes)} de {len(df)}")
grandes.head()
```

Múltiples condiciones

```
# DAP > 30 Y parcela == 1
filtro = df[(df["dap_cm"] > 30) & (df["parcela"] == 1)]
print(f"Grandes en parcela 1: {len(filtro)}")

# Especie específica
coigues = df[df["especie"] == "Nothofagus dombeyi"]
print(f"Coigües: {len(coigues)}")
coigues.describe()
```

Nota sintáctica: en Pandas se usa & (no and) y | (no or), y cada condición va entre paréntesis.

Filtrar con .isin() y .str

```
# Varias especies a la vez
nativas_clave = df[df["especie"].isin(["Nothofagus dombeyi", "Drimys winteri"])]
print(f"Coigüe + Canelo: {len(nativas_clave)}")

# Buscar en texto
df[df["especie"].str.contains("Nothofagus")]
```

Bloque E — Contar y agrupar (20 min)

Conteos de frecuencia

```
# Tabla de frecuencias de especies
df["especie"].value_counts()
```

“Esto es el diccionario de frecuencias de la Sem. 9 — conteo = {}; for obs: conteo[obs] += 1 — en UNA línea.”

```
# Conteo por parcela
df["parcela"].value_counts().sort_index()
```

groupby: la herramienta más poderosa

```
# Estadísticas de DAP por especie
df.groupby("especie")["dap_cm"].mean()
```

“groupby divide los datos en grupos, aplica una función a cada grupo, y combina los resultados. Es el patrón ‘por parcela’ de la Sem. 9, automatizado.”

```
# Múltiples estadísticas por especie
df.groupby("especie")["dap_cm"].agg(["count", "mean", "std", "min", "max"])
```

```
# Estadísticas por parcela
df.groupby("parcela").agg({
    "dap_cm": ["mean", "std"],
    "altura_m": ["mean", "std"],
    "especie": "nunique"          # número de especies únicas
})
```

```
# Tabla cruzada: conteo de especies por parcela
pd.crosstab(df["parcela"], df["especie"])
```

“La tabla cruzada parcela $\times$ especie es la MATRIZ de datos que necesitan para calcular diversidad por parcela. Pandas la genera en una línea.”

Bloque F — Aplicar funciones propias a DataFrames (10 min)

Callback Sem. 10: “Escribieron `shannon_entropy()` la semana pasada. Ahora la aplican sobre datos reales.”

```
import math

def shannon_entropy(abundancias):
    """H' en nats."""
    total = sum(abundancias)
    H = 0
    for n in abundancias:
        if n > 0:
            p = n / total
            H -= p * math.log(p)
    return H

# Obtener abundancias por especie (del dataset completo)
conteos = df["especie"].value_counts()
H_total = shannon_entropy(conteos.values)
print(f"H' del bosque completo: {H_total:.4f} nats")

# H' por parcela - combinando groupby + función propia
for parcela, grupo in df.groupby("parcela"):
    conteos = grupo["especie"].value_counts()
    H = shannon_entropy(conteos.values)
    S = len(conteos)
    print(f"Parcela {parcela}: S={S}, H'={H:.4f}")
```

“El desafío de la Sem. 9 (diversidad por parcela, ~25 líneas manuales), refactorizado con funciones en la Sem. 10 (~10 líneas), ahora con Pandas: ~5 líneas.”

Bloque G — Crear nuevas columnas y exportar (5 min)

```
# Crear una columna calculada
df["area_basal_m2"] = math.pi * (df["dap_cm"] / 200) ** 2
df.head()

# Clasificar con .apply()
def clasificar_tamano(dap):
    if dap > 40: return "grande"
    elif dap > 20: return "mediano"
    else: return "pequeño"

df["tamano"] = df["dap_cm"].apply(clasificar_tamano)
df["tamano"].value_counts()

# Exportar a CSV
df.to_csv("resultado_analisis.csv", index=False)
```

Bloque H — Resumen: la evolución del código (5 min)

Proyectar la misma tarea resuelta en tres semanas:

Semana	Tarea: DAP promedio por especie	Líneas
Sem. 9	Bucle + dict + list comprehension	~12 líneas

Semana	Tarea: DAP promedio por especie	Líneas
Sem. 10	Funciones propias	~8 líneas
Sem. 11	<code>df.groupby("especie")["dap_cm"].mean()</code>	1 línea

“No es que las semanas anteriores fueran inútiles — al contrario. Entienden qué hace esa línea de Pandas PORQUE escribieron los bucles a mano primero.”

PARTE 2: LABORATORIO EN COLAB (90 minutos)

Ejercicio 1: Exploración del dataset forestal (15 min)

Cargar `bosque_valdiviano_simple.csv`. Explorar con `.shape`, `.dtypes`, `.head()`, `.describe()`. Responder: ¿cuántas filas? ¿Cuántas especies? ¿Cuál es el DAP promedio?

Ejercicio 2: Filtrado (15 min)

Filtrar árboles con DAP > 40 cm. Filtrar solo *Nothofagus dombeyi*. Filtrar árboles de parcela 3 con altura > 15 m. ¿Cuántos hay en cada caso?

Ejercicio 3: Conteos y frecuencias (15 min)

`.value_counts()` por especie y por parcela. Tabla cruzada especie × parcela con `pd.crosstab()`. ¿Cuál parcela tiene más especies?

Ejercicio 4: groupby (20 min)

DAP promedio por especie. Altura promedio por parcela. Tabla con count + mean + std por especie. Identificar la especie con mayor DAP promedio.

Ejercicio 5: Aplicar funciones propias (15 min)

Calcular Shannon H' para el dataset completo y para cada parcela por separado. Crear columna `tamano` usando `.apply()` y `clasificar_tamano()`. Contar cuántos árboles grandes/medianos/pequeños hay por parcela.

Ejercicio 6 (desafío): Dataset de cámaras trampa (15 min)

Cargar `camaras_trampa_simulado.csv`. Calcular H' por sitio. ¿Cuál es el sitio más diverso? ¿Cuál tiene más registros? ¿Hay alguna especie que aparezca en todos los sitios excepto uno?

Tarea evaluada #2

Entrega: Semana 12 **Formato:** Notebook Colab **LLMs:** Permitidos con prompt log

Dataset: `camaras_trampa_simulado.csv`

1. Cargar y explorar el dataset (shape, dtypes, head, describe)
2. ¿Cuántos registros por sitio? ¿Por método de muestreo?
3. Tabla cruzada sitio × especie
4. Shannon H' por sitio — ¿cuál es el más diverso?
5. Filtrar solo registros de “cámara trampa” — ¿cambia el ranking de diversidad?
6. Crear una columna `mes` a partir de la columna `fecha` y contar registros por mes
7. (Bonus) Graficar la abundancia por especie de un sitio (anticipando Sem. 12)

Notas pedagógicas

Esta es la clase más gratificante del curso

Los estudiantes van a experimentar el “wow” de Pandas: tareas que tomaban 12 líneas con bucles ahora toman 1 línea. Pero ese “wow” solo funciona porque pasaron por las semanas previas. Si saltaran directo a Pandas, no entenderían qué hace `groupby` — porque nunca escribieron el bucle equivalente a mano.

El error más común: confundir `[]` y `()`

- `df["columna"]` — seleccionar columna (corchetes)
- `df.method()` — llamar método (paréntesis)
- `df[df["x"] > 5]` — filtrar (corchetes con condición dentro)

`groupby` como concepto

`groupby` es la abstracción más importante de Pandas. Conceptualmente es: “divide → aplica → combina” (split-apply-combine). Dedicar tiempo a mostrar que es el mismo patrón que el bucle `for parcela in parcelas: filtrar → calcular → guardar` de la Sem. 9.

Conexión con el resto del curso

Concepto	Dónde se profundiza
<code>pd.read_csv()</code>	Sem. 12 (más opciones de carga), Sem. 16 (examen)
<code>.groupby()</code>	Sem. 12 (agrupaciones complejas, <code>.agg</code> con funciones propias)
<code>.apply()</code>	Sem. 12 (funciones lambda, <code>apply</code> sobre filas)
Filtrado booleano	Sem. 12 (condiciones complejas), Sem. 16 (examen)
Crear columnas	Sem. 12 (transformaciones), Sem. 14 (visualización)
<code>.to_csv()</code>	Sem. 14 (exportar resultados del proyecto)